Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №4

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: «Классификация линейно разделимых образов однослойным персептроном для логических функций n переменных»

Выполнил:

Студент 3 курса

Группы ИИ-26

Кушнеревич Е.А

Проверила:

Андренко К.В.

Брест 2026

Цель работы: Реализовать нейронную сеть (однослойный персептрон) для решения задач синтеза логических функций от n входов. Сравнить эффективность обучения при различных начальных условиях и параметрах шага.

Ход работы:

1. Сгенерировать полную таблицу истинности для заданной логической функции (2^n наборов). На выходе функции должны использоваться значения 0 и 1.
2. Реализовать алгоритм обучения:
- Разделить выборку на обучающую (часть данных) и тестовую (оставшиеся наборы) для проверки обобщающей способности.
- В качестве функции активации использовать сигмоиду
- Провести два эксперимента по обучению:
  А. С фиксированным шагом обучения α
  Б. С адаптивным шагом
- Выход сети интерпретировать как вероятность принадлежности к классу «1».
- Обучение проводить до минимизации суммарной ошибки на обучающей выборке.
3. Провести анализ сходимости:
- Построить графики зависимости суммарной ошибки от номера эпохи для обучающей и тестовой выборок.
- Оценить обобщающую способность: корректно ли сеть предсказывает значения на тех наборах из таблицы истинности, которые она не видела при обучении.
- Зафиксировать количество эпох, необходимых для настройки весов при разных значениях α
4. Вывести итоговые значения весовых коэффициентов и порога.
5. Реализовать режим функционирования:
- Программа должна принимать на вход вектор из n чисел (0 или 1).
- Сеть должна мгновенно выдавать результат (вероятность и округленный класс).
6. Сделать вывод о способности сигмоидального персептрона представлять базовые логические операции и восстанавливать полную таблицу истинности по частичной выборке.

№ of variant	n	Task
8	6	AND^i

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt
from itertools import product

n = 6

def logic_func(row):
    x = list(row)
    return int((1 - x[0]) * x[1] * x[2] * x[3] * x[4] * x[5])


all_inputs = np.array(list(product([0, 1], repeat = n)))
all_outputs = np.array([logic_func(row) for row in all_inputs]).reshape(-1, 1)

print("Таблица истинности:")
```

```
print(f"{'Входы':<20} {'Выход'}")
for row, out in zip(all_inputs, all_outputs):
    print(f" {row.astype(int)}  ->  {int(out[0])}")

ones_idx  = np.where(all_outputs[:, 0] == 1)[0]
zeros_idx = np.where(all_outputs[:, 0] == 0)[0]

np.random.seed(42)
zeros_perm = np.random.permutation(zeros_idx)
split = int(0.75 * len(zeros_idx))

train_idx = np.concatenate([ones_idx, zeros_perm[:split]])
test_idx  = zeros_perm[split:]

X_train, y_train = all_inputs[train_idx], all_outputs[train_idx]
X_test,  y_test  = all_inputs[test_idx],  all_outputs[test_idx]

print(f"\nОбучающая выборка ({len(train_idx)} наборов): индексы
{sorted(train_idx.tolist())}")
print(f"Тестовая выборка  ({len(test_idx)} наборов): индексы
{sorted(test_idx.tolist())}")

def sigmoid(s):
    return 1.0 / (1.0 + np.exp(-np.clip(s, -500, 500)))

def linear_output(W, X, T):
    return X @ W - T

def bce_loss(y_pred, y_true):
    eps = 1e-12
    yp = np.clip(y_pred, eps, 1 - eps)
    return -np.sum(y_true * np.log(yp) + (1 - y_true) * np.log(1 - yp))

def fixed_fit(X_tr, y_tr, X_te, y_te, alpha=0.1, max_epochs=10000, Ee=0.05):
    N, n = X_tr.shape
    W = np.zeros((n, 1))
    T = 0.0
    Es_train, Es_test = [], []
    stopped = max_epochs

    print(f"\n[Фиксированный] alpha={alpha}, Ee={Ee}")

    for ep in range(max_epochs):
        order = np.random.permutation(N)
        for idx in order:
            xi = X_tr[idx:idx+1]
            ti = float(y_tr[idx, 0])
            s  = float(linear_output(W, xi, T)[0, 0])
            yi = sigmoid(s)
            d  = yi - ti
            W -= alpha * xi.T * d
            T += alpha * d

        p_tr = sigmoid(linear_output(W, X_tr, T))
```

```
        es_tr = bce_loss(p_tr, y_tr)
        Es_train.append(es_tr)

        p_te = sigmoid(linear_output(W, X_te, T))
        es_te = bce_loss(p_te, y_te)
        Es_test.append(es_te)

        if ep % 500 == 0 and ep > 0:
            print(f" эпоха {ep:5d} | train={es_tr:.5f} | test={es_te:.5f}")

        if es_tr <= Ee:
            stopped = ep + 1
            print(f" Остановка на эпохе {stopped}, train = {es_tr:.5f}\n")
            break
    else:
        print(f" Лимит {max_epochs} эпох\n")

    return W, T, Es_train, Es_test, stopped

def adaptive_fit(X_tr, y_tr, X_te, y_te, max_epochs=10000, Ee=0.05):
    N, n = X_tr.shape
    W = np.zeros((n, 1))
    T = 0.0
    Es_train, Es_test = [], []
    stopped = max_epochs

    print(f"[Адаптивный] α(t), Ee={Ee}")

    for ep in range(max_epochs):
        order = np.random.permutation(N)
        for idx in order:
            xi = X_tr[idx:idx+1]
            ti = float(y_tr[idx, 0])
            norm_sq = float(np.sum(xi ** 2))
            alpha_t = 1.0 / norm_sq if norm_sq > 1e-12 else 0.1

            s = float(linear_output(W, xi, T)[0, 0])
            yi = sigmoid(s)
            d = yi - ti
            W -= alpha_t * xi.T * d
            T += alpha_t * d

        p_tr = sigmoid(linear_output(W, X_tr, T))
        es_tr = bce_loss(p_tr, y_tr)
        Es_train.append(es_tr)

        p_te = sigmoid(linear_output(W, X_te, T))
        es_te = bce_loss(p_te, y_te)
        Es_test.append(es_te)

        if ep % 500 == 0 and ep > 0:
            print(f"  эпоха {ep:5d} | train={es_tr:.5f} | test={es_te:.5f}")

        if es_tr <= Ee:
```

```
            stopped = ep + 1
            print(f"  Остановка на эпохе {stopped}, train={es_tr:.5f}\n")
            break
    else:
        print(f"  Лимит {max_epochs} эпох\n")

    return W, T, Es_train, Es_test, stopped

Ee = 0.01
max_epochs = 10000

np.random.seed(42)
W_fix, T_fix, es_tr_fix, es_te_fix, ep_fix = fixed_fit(
    X_train, y_train, X_test, y_test, alpha=0.1, Ee=Ee, max_epochs = max_epochs)

np.random.seed(42)
W_adp, T_adp, es_tr_adp, es_te_adp, ep_adp = adaptive_fit(
    X_train, y_train, X_test, y_test, Ee=Ee, max_epochs = max_epochs)

print("Итоговые веса и порог:")
print(f" Фиксированный: W={W_fix.flatten()}, T={T_fix:.5f}, эпох={ep_fix}")
print(f" Адаптивный: W={W_adp.flatten()}, T={T_adp:.5f}, эпох={ep_adp}")

print("\nОценка на тестовой выборке (фиксированный):")
for xi, ti in zip(X_test, y_test):
    s = float(linear_output(W_fix, xi.reshape(1,-1), T_fix)[0, 0])
    prob = sigmoid(s)
    pred = 1 if prob >= 0.5 else 0
    ok   = "Yay" if pred == int(ti[0]) else "X"
    print(f"  {xi.astype(int)} -> ŷ = {prob:.6f} -> класс {pred} (эталон
{int(ti[0])}) {ok}")

print("\nОценка на тестовой выборке (адаптивный):")
for xi, ti in zip(X_test, y_test):
    s = float(linear_output(W_adp, xi.reshape(1,-1), T_adp)[0, 0])
    prob = sigmoid(s)
    pred = 1 if prob >= 0.5 else 0
    ok = "Yay" if pred == int(ti[0]) else "X"
    print(f"{xi.astype(int)} -> ŷ={prob:.6f} -> класс {pred} (эталон {int(ti[0])})
{ok}")

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for ax, es_tr, es_te, ep, title, alpha_lbl in [
    (axes[0], es_tr_fix, es_te_fix, ep_fix, "Фиксированный шаг α=0.1", "α=0.1"),
    (axes[1], es_tr_adp, es_te_adp, ep_adp, "Адаптивный шаг α(t)", "α(t)"),
]:
    epochs = np.arange(1, len(es_tr) + 1)
    ax.plot(epochs, es_tr, lw=2, color="#2563EB", label="Обучающая Es")
    ax.plot(epochs, es_te, lw=2, color="#DC2626", ls="--", label="Тестовая Es")
    ax.axhline(Ee, color="gray", lw=1.3, ls=":", label=f"Eₑ = {Ee}")
    ax.set_yscale("log")
    ax.set_xlabel("Эпоха", fontsize=11)
    ax.set_ylabel("BCE Loss", fontsize=11)
```

```
    ax.set_title(f"{title}", fontsize=12, fontweight='bold')
    ax.legend(fontsize=10)
    ax.grid(alpha=0.35)

plt.tight_layout()
plt.show()

print("\n" + "—" * 50)
print(f" Режим функционирования (модель: адаптивный)")
print(f" Введите {n} чисел (0 или 1) через пробел, или 'exit'\n")

while True:
    user_input = input(" -> ").strip()
    if user_input in ("exit", ""):
        break
    try:
        vals = list(map(int, user_input.split()))
        assert len(vals) == n and all(v in (0,1) for v in vals)

        point = np.array(vals, dtype=float).reshape(1, -1)
        s = float(linear_output(W_adp, point, T_adp)[0, 0])
        prob = sigmoid(s)
        cls = 1 if prob >= 0.5 else 0

        print(f"S = {s:+.5f}")
        print(f"ŷ = {prob:.5f}  (вероятность класса 1)")
        print(f"Класс = {cls}\n")

    except Exception:
        print(f" Ошибка. Введите ровно {n} чисел (0 или 1)\n")

print("Работа завершена.")
```

Результат тестирования:

Таблица истинности:

Входы		Выход
[0 0 0 0 0 0]	->	0
[0 0 0 0 0 1]	->	0
[0 0 0 0 1 0]	->	0
[0 0 0 0 1 1]	->	0
[0 0 0 1 0 0]	->	0
[0 0 0 1 0 1]	->	0
[0 0 0 1 1 0]	->	0
[0 0 0 1 1 1]	->	0
[0 0 1 0 0 0]	->	0
[0 0 1 0 0 1]	->	0
[0 0 1 0 1 0]	->	0
[0 0 1 0 1 1]	->	0
[0 0 1 1 0 0]	->	0
[0 0 1 1 0 1]	->	0
[0 0 1 1 1 0]	->	0
[0 0 1 1 1 1]	->	0
[0 1 0 0 0 0]	->	0
[0 1 0 0 0 1]	->	0
[0 1 0 0 1 0]	->	0
[0 1 0 0 1 1]	->	0

```
  [0 1 0 1 0 0]  ->  0
  [0 1 0 1 0 1]  ->  0
  [0 1 0 1 1 0]  ->  0
  [0 1 0 1 1 1]  ->  0
  [0 1 1 0 0 0]  ->  0
  [0 1 1 0 0 1]  ->  0
  [0 1 1 0 1 0]  ->  0
  [0 1 1 0 1 1]  ->  0
  [0 1 1 1 0 0]  ->  0
  [0 1 1 1 0 1]  ->  0
  [0 1 1 1 1 0]  ->  0
  [0 1 1 1 1 1]  ->  1
  [1 0 0 0 0 0]  ->  0
  [1 0 0 0 0 1]  ->  0
  [1 0 0 0 1 0]  ->  0
  [1 0 0 0 1 1]  ->  0
  [1 0 0 1 0 0]  ->  0
  [1 0 0 1 0 1]  ->  0
  [1 0 0 1 1 0]  ->  0
  [1 0 0 1 1 1]  ->  0
  [1 0 1 0 0 0]  ->  0
  [1 0 1 0 0 1]  ->  0
  [1 0 1 0 1 0]  ->  0
  [1 0 1 0 1 1]  ->  0
  [1 0 1 1 0 0]  ->  0
  [1 0 1 1 0 1]  ->  0
  [1 0 1 1 1 0]  ->  0
  [1 0 1 1 1 1]  ->  0
  [1 1 0 0 0 0]  ->  0
  [1 1 0 0 0 1]  ->  0
  [1 1 0 0 1 0]  ->  0
  [1 1 0 0 1 1]  ->  0
  [1 1 0 1 0 0]  ->  0
  [1 1 0 1 0 1]  ->  0
  [1 1 0 1 1 0]  ->  0
  [1 1 0 1 1 1]  ->  0
  [1 1 1 0 0 0]  ->  0
  [1 1 1 0 0 1]  ->  0
  [1 1 1 0 1 0]  ->  0
  [1 1 1 0 1 1]  ->  0
  [1 1 1 1 0 0]  ->  0
  [1 1 1 1 0 1]  ->  0
  [1 1 1 1 1 0]  ->  0
  [1 1 1 1 1 1]  ->  0
```

Обучающая выборка (48 наборов): индексы [0, 1, 2, 3, 4, 5, 6, 8, 9, 11, 12, 13, 15, 16, 17, 19, 21, 24, 25, 26, 27, 29, 30, 31, 32, 33, 34, 35, 37, 38, 41, 42, 44, 45, 46, 47, 48, 49, 50, 54, 55, 56, 57, 58, 59, 60, 61, 62]
Тестовая выборка (16 наборов): индексы [7, 10, 14, 18, 20, 22, 23, 28, 36, 39, 40, 43, 51, 52, 53, 63]

[Фиксированный] alpha=0.1, Ee=0.01

```
  эпоха   500 | train=0.93255 | test=0.80099
  эпоха  1000 | train=0.56644 | test=1.04344
  эпоха  1500 | train=0.40743 | test=1.24345
  эпоха  2000 | train=0.31762 | test=1.39389
  эпоха  2500 | train=0.26005 | test=1.53566
  эпоха  3000 | train=0.22000 | test=1.66068
  эпоха  3500 | train=0.19053 | test=1.77058
  эпоха  4000 | train=0.16796 | test=1.86753
  эпоха  4500 | train=0.15013 | test=1.95431
  эпоха  5000 | train=0.13570 | test=2.03718
  эпоха  5500 | train=0.12378 | test=2.11783
```

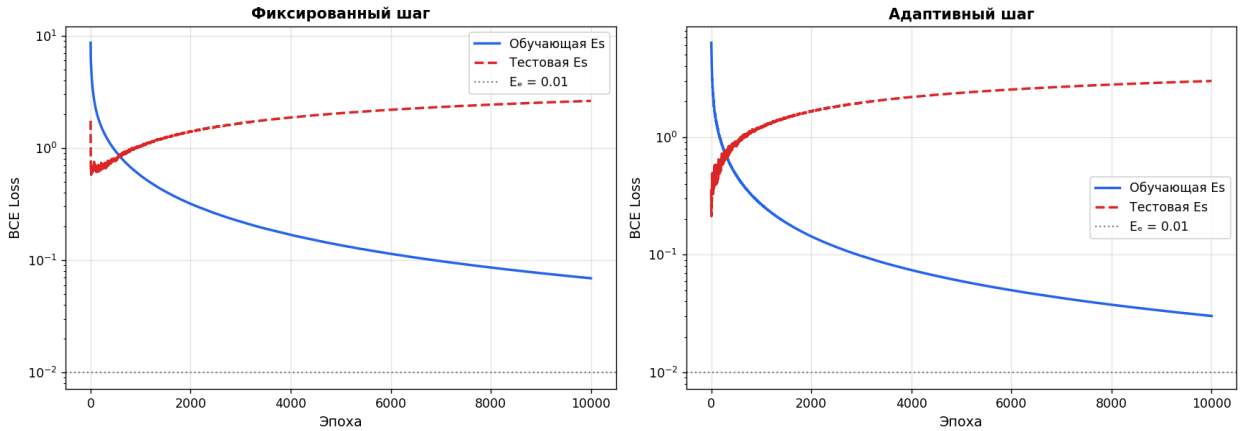
```
  эпоха  6000 | train=0.11377 | test=2.18600
  эпоха  6500 | train=0.10525 | test=2.25154
  эпоха  7000 | train=0.09791 | test=2.31457
  эпоха  7500 | train=0.09152 | test=2.37415
  эпоха  8000 | train=0.08590 | test=2.42848
  эпоха  8500 | train=0.08093 | test=2.47989
  эпоха  9000 | train=0.07651 | test=2.52894
  эпоха  9500 | train=0.07254 | test=2.57827
 Лимит 10000 эпох

[Адаптивный] α(t), Ee=0.01
  эпоха   500 | train=0.47333 | test=0.88776
  эпоха  1000 | train=0.26771 | test=1.22132
  эпоха  1500 | train=0.18642 | test=1.47180
  эпоха  2000 | train=0.14332 | test=1.65149
  эпоха  2500 | train=0.11617 | test=1.81729
  эпоха  3000 | train=0.09764 | test=1.96007
  эпоха  3500 | train=0.08422 | test=2.08338
  эпоха  4000 | train=0.07405 | test=2.19110
  эпоха  4500 | train=0.06608 | test=2.28647
  эпоха  5000 | train=0.05964 | test=2.37664
  эпоха  5500 | train=0.05431 | test=2.46367
  эпоха  6000 | train=0.04989 | test=2.53686
  эпоха  6500 | train=0.04613 | test=2.60683
  эпоха  7000 | train=0.04288 | test=2.67376
  эпоха  7500 | train=0.04006 | test=2.73673
  эпоха  8000 | train=0.03760 | test=2.79401
  эпоха  8500 | train=0.03542 | test=2.84803
  эпоха  9000 | train=0.03348 | test=2.89940
  эпоха  9500 | train=0.03173 | test=2.95090
  Лимит 10000 эпох

Итоговые веса и порог:
 Фиксированный: W=[-8.79633833  8.16180712  0.9214709   8.50100326  8.50063269  8.16133834], T=30.79030, эпох=10000
 Адаптивный: W=[-9.87447486  9.79746187  1.2261971  10.12885816 10.12868297  9.79728333], T=36.91192, эпох=10000

Оценка на тестовой выборке (фиксированный):
  [1 1 0 0 1 1] -> ŷ = 0.000000 -> класс 0 (эталон 0) Yay
  [1 0 1 0 0 0] -> ŷ = 0.000000 -> класс 0 (эталон 0) Yay
  [1 0 0 1 0 0] -> ŷ = 0.000000 -> класс 0 (эталон 0) Yay
  [0 1 0 1 1 1] -> ŷ = 0.926524 -> класс 1 (эталон 0) X
  [1 1 0 1 0 1] -> ŷ = 0.000000 -> класс 0 (эталон 0) Yay
  [0 0 1 0 1 0] -> ŷ = 0.000000 -> класс 0 (эталон 0) Yay
  [0 1 0 1 1 0] -> ŷ = 0.003587 -> класс 0 (эталон 0) Yay
  [0 1 0 0 1 0] -> ŷ = 0.000001 -> класс 0 (эталон 0) Yay
  [1 1 1 1 1 1] -> ŷ = 0.004771 -> класс 0 (эталон 0) Yay
  [0 1 0 1 0 0] -> ŷ = 0.000001 -> класс 0 (эталон 0) Yay
  [0 0 0 1 1 1] -> ŷ = 0.003585 -> класс 0 (эталон 0) Yay
  [1 0 1 0 1 1] -> ŷ = 0.000000 -> класс 0 (эталон 0) Yay
  [0 0 1 1 1 0] -> ŷ = 0.000003 -> класс 0 (эталон 0) Yay
  [0 1 1 1 0 0] -> ŷ = 0.000002 -> класс 0 (эталон 0) Yay
  [1 1 0 1 0 0] -> ŷ = 0.000000 -> класс 0 (эталон 0) Yay
  [1 0 0 1 1 1] -> ŷ = 0.000001 -> класс 0 (эталон 0) Yay

Оценка на тестовой выборке (адаптивный):
  [1 1 0 0 1 1] -> ŷ=0.000000 -> класс 0 (эталон 0) Yay
  [1 0 1 0 0 0] -> ŷ=0.000000 -> класс 0 (эталон 0) Yay
  [1 0 0 1 0 0] -> ŷ=0.000000 -> класс 0 (эталон 0) Yay
  [0 1 0 1 1 1] -> ŷ=0.949806 -> класс 1 (эталон 0) X
  [1 1 0 1 0 1] -> ŷ=0.000000 -> класс 0 (эталон 0) Yay
  [0 0 1 0 1 0] -> ŷ=0.000000 -> класс 0 (эталон 0) Yay
```

[0 1 0 1 1 0] -> ŷ=0.001051 -> класс 0 (эталон 0) Yay
[0 1 0 0 1 0] -> ŷ=0.000000 -> класс 0 (эталон 0) Yay
[1 1 1 1 1 1] -> ŷ=0.003309 -> класс 0 (эталон 0) Yay
[0 1 0 1 0 0] -> ŷ=0.000000 -> класс 0 (эталон 0) Yay
[0 0 0 1 1 1] -> ŷ=0.001051 -> класс 0 (эталон 0) Yay
[1 0 1 0 1 1] -> ŷ=0.000000 -> класс 0 (эталон 0) Yay
[0 0 1 1 1 0] -> ŷ=0.000000 -> класс 0 (эталон 0) Yay
[0 1 1 1 0 0] -> ŷ=0.000000 -> класс 0 (эталон 0) Yay
[1 1 0 1 0 0] -> ŷ=0.000000 -> класс 0 (эталон 0) Yay
[1 0 0 1 1 1] -> ŷ=0.000000 -> класс 0 (эталон 0) Yay

Режим функционирования (модель: адаптивный)
Введите 6 чисел (0 или 1) через пробел, или 'exit'

-> 1 1 1 1 1 1
S = -5.70791
ŷ = 0.00331 (вероятность класса 1)
Класс = 0

График сходимостей:



Вывод: Проделав данную лабораторную работу я реализовал нейронную сеть (однослойный персептрон) для решения задач синтеза логических функций от n входов. Сравнил эффективность обучения при различных начальных условиях и параметрах шага.